IN a real time means in a company we do this type of code

In [4]:
```
import pandas as pd   # for DF
import numpy as np    # for array operations
from sklearn.model_selection import train_test_split # foe train , test
from sklearn.linear_model import LinearRegression    # for model
from sklearn.metrics import mean_squared_error,root_mean_squared_error  # for matri
```

In [5]:
```
from sklearn.datasets import fetch_california_housing
df = fetch_california_housing(as_frame= True).frame

x = df.drop(columns = ['MedHouseVal'])
y = df['MedHouseVal']

x_train, x_test ,y_train ,y_test = train_test_split(x,y ,
                                                    test_size= 0.2,
                                                    random_state = 1234)
lr = LinearRegression()
lr.fit(x_train,y_train)
```

Out[5]:

▾ LinearRegression

▸ Parameters

In [6]:
```
y_test_pred = lr.predict(x_test)
MSE = mean_squared_error(y_test,y_test_pred)
RMSE = root_mean_squared_error(y_test,y_test_pred)
MSE,RMSE
```

Out[6]: (0.5322341499776327, 0.7295437957913375)

HOW TO SAVE AND LOAD MODEL

- This method save only model

In [7]:
```
# now data is ready , so need to save it and load it
# Save the model ) {V.IMP}
# model save krna uske baad client ko dena
import pickle

file = open(r'lr_california','wb')
dump_model = pickle.dump(lr,file)                    #pickle( model name , file name
```

In [8]:
```
# how to loaded model
file = open(r'lr_california.pkl','rb')
loaded_model = pickle.load(file)
```

```
loaded_model['model']
```

Out[8]:

▾ LinearRegression

▸ Parameters

In [ ]:

THIS IS A BEST METHOD TO SAVE ADN LOAD A MODEL

- This method not save only mode but it save all the data (like linear model, x_train & y_train data, columns,)

In [33]:
```
# i want to save all the data like linear model, data x_trian & y_train, number of
# it is saving
file1= open('lr_california.pkl','wb')
d1= {'model':lr, 'columns':x_test.columns} # 1st create a dictionay / d is a full d
pickle.dump(d1,file1)
```

In [34]:
```
# now load it how to load
file1 = open('lr_california.pkl','rb')
ans = pickle.load(file1)
ans['model'].predict([[1,2,3,4,5,6,7,8]])
```

```
C:\Users\q\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning
: X does not have valid feature names, but LinearRegression was fitted with feature
names
  warnings.warn(
```

Out[34]:
```
array([-39.94831177])
```

In [ ]:
```
# Q1) what we are saving in 'lr_california.pkl'
# ans= we are saving model and columns
# Q2) in which format = in a dictionary format
```

In [ ]:

Observation no-1

In [28]:
```
# what is a model name :- linear regression
# in your model name What is a variable name :- lr
# lr train with haw many columns :- 8 input columns ( 8+1=o/p )
# whenever, if you handover this model to client:-
#                                                client should pass only 8 values a
# i need to pass values with 2D means a double list[[]]

lr.predict([10,20,30,40,50,60,70,80]) #error
```

```
C:\Users\q\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning
: X does not have valid feature names, but LinearRegression was fitted with feature
names
  warnings.warn(
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[28], line 7
      1 # what is a model name :- linear regression
      2 # in your model name What is a variable name :- lr
      3 # lr train with haw many columns :- 8 input columns ( 8+1=o/p )
      4 # whenever, if you handover this model to client:-
      5 #                                                 client should pass only 8 v
alues at a time [8 values means i/p]
      6 # i need to pass values with 2D means a double list[[]]
----> 7 lr.predict([10,20,30,40,50,60,70,80])

File ~\anaconda3\Lib\site-packages\sklearn\linear_model\_base.py:298, in LinearMode
l.predict(self, X)
    284 def predict(self, X):
    285     """
    286     Predict using the linear model.
    287
   (...)    296         Returns predicted values.
    297     """
--> 298     return self._decision_function(X)

File ~\anaconda3\Lib\site-packages\sklearn\linear_model\_base.py:277, in LinearMode
l._decision_function(self, X)
    274 def _decision_function(self, X):
    275     check_is_fitted(self)
--> 277     X = validate_data(self, X, accept_sparse=["csr", "csc", "coo"], reset=Fa
lse)
    278     coef_ = self.coef_
    279     if coef_.ndim == 1:

File ~\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2954, in validate_dat
a(_estimator, X, y, reset, validate_separately, skip_check_array, **check_params)
   2952         out = X, y
   2953 elif not no_val_X and no_val_y:
-> 2954     out = check_array(X, input_name="X", **check_params)
   2955 elif no_val_X and not no_val_y:
   2956     out = _check_y(y, **check_params)

File ~\anaconda3\Lib\site-packages\sklearn\utils\validation.py:1091, in check_array
(array, accept_sparse, accept_large_sparse, dtype, order, copy, force_writeable, for
ce_all_finite, ensure_all_finite, ensure_non_negative, ensure_2d, allow_nd, ensure_m
in_samples, ensure_min_features, estimator, input_name)
   1084         else:
   1085             msg = (
   1086                 f"Expected 2D array, got 1D array instead:\narray={array}.
\n"
   1087                 "Reshape your data either using array.reshape(-1, 1) if "
   1088                 "your data has a single feature or array.reshape(1, -1) "
   1089                 "if it contains a single sample."
   1090             )
-> 1091         raise ValueError(msg)
   1093 if dtype_numeric and hasattr(array.dtype, "kind") and array.dtype.kind in "U
SV":
   1094     raise ValueError(
   1095         "dtype='numeric' is not compatible with arrays of bytes/strings."
```

```
   1096         "Convert your data to numeric values explicitly instead."
   1097     )
```

```
ValueError: Expected 2D array, got 1D array instead:
array=[10 20 30 40 50 60 70 80].
Reshape your data either using array.reshape(-1, 1) if your data has a single featur
e or array.reshape(1, -1) if it contains a single sample.
```

In [29]:
```
# i need to pass values with 2D means a double list[[]]
lr.predict([[10,20,30,40,50,60,70,80]])
```

```
C:\Users\q\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning
: X does not have valid feature names, but LinearRegression was fitted with feature
names
  warnings.warn(
```

Out[29]:
```
array([-73.66671914])
```

In [30]:
```
# if you pass only 7 values
lr.predict([10,20,30,40,50,60,70])
# error model train 8 columns but passes only 7 columns
```

```
C:\Users\q\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning
: X does not have valid feature names, but LinearRegression was fitted with feature
names
  warnings.warn(
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[30], line 2
      1 # if you pass only 7 values
----> 2 lr.predict([10,20,30,40,50,60,70])

File ~\anaconda3\Lib\site-packages\sklearn\linear_model\_base.py:298, in LinearMode
l.predict(self, X)
    284 def predict(self, X):
    285     """
    286     Predict using the linear model.
    287
   (...)    296         Returns predicted values.
    297     """
--> 298     return self._decision_function(X)

File ~\anaconda3\Lib\site-packages\sklearn\linear_model\_base.py:277, in LinearMode
l._decision_function(self, X)
    274 def _decision_function(self, X):
    275     check_is_fitted(self)
--> 277     X = validate_data(self, X, accept_sparse=["csr", "csc", "coo"], reset=Fa
lse)
    278     coef_ = self.coef_
    279     if coef_.ndim == 1:

File ~\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2954, in validate_dat
a(_estimator, X, y, reset, validate_separately, skip_check_array, **check_params)
   2952         out = X, y
   2953 elif not no_val_X and no_val_y:
-> 2954     out = check_array(X, input_name="X", **check_params)
   2955 elif no_val_X and not no_val_y:
   2956     out = _check_y(y, **check_params)

File ~\anaconda3\Lib\site-packages\sklearn\utils\validation.py:1091, in check_array
(array, accept_sparse, accept_large_sparse, dtype, order, copy, force_writeable, for
ce_all_finite, ensure_all_finite, ensure_non_negative, ensure_2d, allow_nd, ensure_m
in_samples, ensure_min_features, estimator, input_name)
   1084         else:
   1085             msg = (
   1086                 f"Expected 2D array, got 1D array instead:\narray={array}.
\n"
   1087                 "Reshape your data either using array.reshape(-1, 1) if "
   1088                 "your data has a single feature or array.reshape(1, -1) "
   1089                 "if it contains a single sample."
   1090             )
-> 1091         raise ValueError(msg)
   1093 if dtype_numeric and hasattr(array.dtype, "kind") and array.dtype.kind in "U
SV":
   1094     raise ValueError(
   1095         "dtype='numeric' is not compatible with arrays of bytes/strings."
   1096         "Convert your data to numeric values explicitly instead."
   1097     )

ValueError: Expected 2D array, got 1D array instead:
array=[10 20 30 40 50 60 70].
Reshape your data either using array.reshape(-1, 1) if your data has a single featur
```

```
e or array.reshape(1, -1) if it contains a single sample.
```

In [35]:
```
# Q) why we are passing 2D data
# ans:- beacuse x_test is 2D data , so we  need to provide data in 2D
lr.predict(x_test)
x_test.ndim
# because my x_test is not a single values = 4128
4128 rows im passing at a time , each row has 8 values
```

In [ ]:
```
# if we pass one sample --->this is called real time or real end point prediction
# if we pass multiple inputs ---> this is called batch prediction
```

In [ ]:
```

```

Obesrvation no - 2, (Save the model) {V.IMP}

- model save krna uske baad client ko dena

- we need to save the model :- we need to save **lr**

- there are 2 packages only
    - 1. pickle format
    - 2. joblib format (generally joblib is a suitable)

In [38]:
```
# path format :- whwre is the location first (C,D or E drive)  ---> 1
#                what is the file name ---> 2
#                what is the extension ---> 3

# where is the location :- same loation where python avaible
# how can got it --
# suppose filename:- lr_california
# extenstion :- .pkl (this is a pickle file)

# path :- (r'lr.california.pkl')

# i have a import pickle
```

In [ ]:
```
under the pickle
    - we have a
    - 1) dump means== save == writing == (wb) and
    - 2) load means == open == reading == (rb) is there
```

pickle

In [11]:
```
#now data is ready , we need to save it
# how save the data
import pickle

file = open(r'lr_california.pkl','wb')
pickle.dump(lr,file)  #0B ------------------> it is a 0B so it gives a problem
```

In [12]:
```
with open(r'lr_california.pkl','wb') as file :
    pickle.dump(lr,file)                        # now it a 724 kb so it save succes

# it is same like this
# with open(r"data.txt",'wb') as file:
#     file.write("hii how are you")
```

In [13]:
```
# how to loaded model
file = open(r'lr_california.pkl','rb')
loaded_model = pickle.load(file)
loaded_model
```

Out[13]:
```
▾ LinearRegression i ?

▸ Parameters
```

In [20]:
```
# i want to save all the data like linear model, data x_trian & y_train, number of
# it is saving
file1= open('lr_california.pkl','wb')
d1= {'model':lr, 'columns':x_test.columns} # 1st create a dictionay / d is a full d
pickle.dump(d1,file1)
```

In [ ]:
```
# Q1) what we are saving in 'lr_california.pkl'
# ans= we are saving model and columns
# Q2) in which format = in a dictionary format
```

In [26]:
```
# now load it how to load
file1 = open('lr_california.pkl','rb')
ans = pickle.load(file1)
ans['model'].predict([[1,2,3,4,5,6,7,8]])
```

```
C:\Users\q\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning
: X does not have valid feature names, but LinearRegression was fitted with feature
names
  warnings.warn(
```

Out[26]:
```
array([-39.94831177])
```

In [ ]:

In [ ]:
```
# summmery what we did yesterday

# stp1:- read the california data
# stp2:- seperate x and y
# stp3:- divide into train test split
# stp4:- develop a model
```

```
# stp5:- metrics
# stp6:- save the model in pickle ,dict format
# stp7:- load the dictionary
# stp8:- pass the unseen data
```

joblib

In [2]:
```
# joblib same like a pickle package to import ans use
import pickle --- import joblib
pickle.dump   --- joblib.dump
pickle.load   --- joblib.load

# but in a pickle we use open first ,
# but in a joblib we use directly
```

In [9]:
```
import joblib
d = {"model":lr,"columns":x_test.columns}

joblib.dump(d,'california.joblib') #---> save the file
```

Out[9]:
```
['california.joblib']
```

In [11]:
```
loaded_model = joblib.load('california.joblib')
loaded_model['model'].predict([[10,20,30,40,50,60,70,80]])
```

```
C:\Users\q\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning
: X does not have valid feature names, but LinearRegression was fitted with feature
names
  warnings.warn(
```

Out[11]:
```
array([-73.66671914])
```

In [ ]:
```

```